

Department of Computer Engineering COMSATS
University Islamabad, Attock Campus

Lab Rubrics Form	
Lab Name	Aliasing Demonstration using Sine Waves at same Sampling Frequency. Fourier Series Approximation of a Square Wave.
Lab Number	02
Submitted by:	
Student Names	Registration #
Sumaira Safeer	FA22-BCE-019

Rubrics		Marks		
		1	2	3
PLO-5 Modern Tools Usage	Ability to execute the experiment utilizing hardware, software tools, or a combination of both.			
PLO-10 Communication Skills	Present lab activities effectively through detailed lab reports, oral examinations, and engaging presentations.			

Lab-02

Aliasing Demonstration using Sine Waves at same Sampling Frequency, And Fourier Series Approximation of a Square Wave.

Objective:

- To understand the concept of aliasing in signal sampling and a square wave using Fourier Series expansion.
- To show that different continuous-time signals (1 kHz and 3 kHz) can become indistinguishable after sampling when the sampling frequency is too low.
- To visually demonstrate through MATLAB plots that different signals may produce identical discrete-time sequences when undersampled.
- To approximate the square wave by summing sinusoidal harmonics. And study the effect of increasing the number of harmonics ($N=1,3,5$) on the accuracy of the approximation.

Lab Task:

1. Aliasing Demonstration:

- Generate a continuous time 1 kHz sine wave call it $x_1(t)$. Sample at $f_s=2$ kHz.
- Generate a continuous time 3kHz sine wave call it $x_2(t)$. Sample at $f_s=2$ kHz.
- Show by plotting that the discrete time signals are same even though continuous time signals are different.

Code:

```
clear;
close all;
fs = 2000;      % Sampling frequency (Hz)
Ts = 1/fs;      % Sampling period (s)
t_cont = 0:1e-5:0.005; % Continuous time vector (0 to 5 ms)
n = 0:9;        % Discrete sample indices (for first few samples)

f1 = 1000;      % Frequency of signal 1 (Hz)
f2 = 3000;      % Frequency of signal 2 (Hz)

% Define continuous-time and discrete-time signals
x1_t = sin(2*pi*f1*t_cont); % Continuous 1 kHz
x2_t = sin(2*pi*f2*t_cont); % Continuous 3 kHz

x1_n = sin(2*pi*f1*n*Ts);    % Sampled 1 kHz
```

```

x2_n = sin(2*pi*f2*n*Ts);    % Sampled 3 kHz (Aliased)

% Optional: remove floating-point noise
x1_n = round(x1_n, 10);
x2_n = round(x2_n, 10);

% Define axes limits and ticks
x_ticks = 0:0.5:5;    % Time ticks (ms)
y_ticks = -1:0.5:1;    % Amplitude ticks
x_axis_ms = t_cont * 1000; % convert time to milliseconds

% 1 kHz Signal
figure('Name','1 kHz Sampling Demonstration','NumberTitle','off');

subplot(3,1,1);
plot(x_axis_ms, x1_t, 'b', 'LineWidth', 1.5);
title('Continuous-Time Signal  $x_1(t) = \sin(2\pi \cdot 1000 \cdot t)$ ');
xlabel('Time (ms)'); ylabel('Amplitude');
grid on; ylim([-1.2 1.2]); yticks(y_ticks);
xlim([0 5]); xticks(x_ticks);

subplot(3,1,2);
stem(n*Ts*1000, x1_n, 'ro', 'filled');
title('Sampled Signal  $x_1[n]$  ( $f_s = 2$  kHz)');
xlabel('Time (ms)'); ylabel('Amplitude');
grid on; ylim([-1.2 1.2]); yticks(y_ticks);
xlim([0 5]); xticks(x_ticks);

subplot(3,1,3);
plot(x_axis_ms, x1_t, 'b', 'LineWidth', 1.2); hold on;
stem(n*Ts*1000, x1_n, 'ro', 'filled');
title('Overlay: Continuous + Sampled Signal (1 kHz)');
xlabel('Time (ms)'); ylabel('Amplitude');
legend('Continuous', 'Samples');
grid on; ylim([-1.2 1.2]); yticks(y_ticks);
xlim([0 5]); xticks(x_ticks);

% 3 kHz Signal - Aliasing Occurs
figure('Name','3 kHz Sampling Demonstration','NumberTitle','off');

subplot(3,1,1);
plot(x_axis_ms, x2_t, 'g', 'LineWidth', 1.5);
title('Continuous-Time Signal  $x_2(t) = \sin(2\pi \cdot 3000 \cdot t)$ ');
xlabel('Time (ms)'); ylabel('Amplitude');
grid on; ylim([-1.2 1.2]); yticks(y_ticks);
xlim([0 5]); xticks(x_ticks);

subplot(3,1,2);

```

```

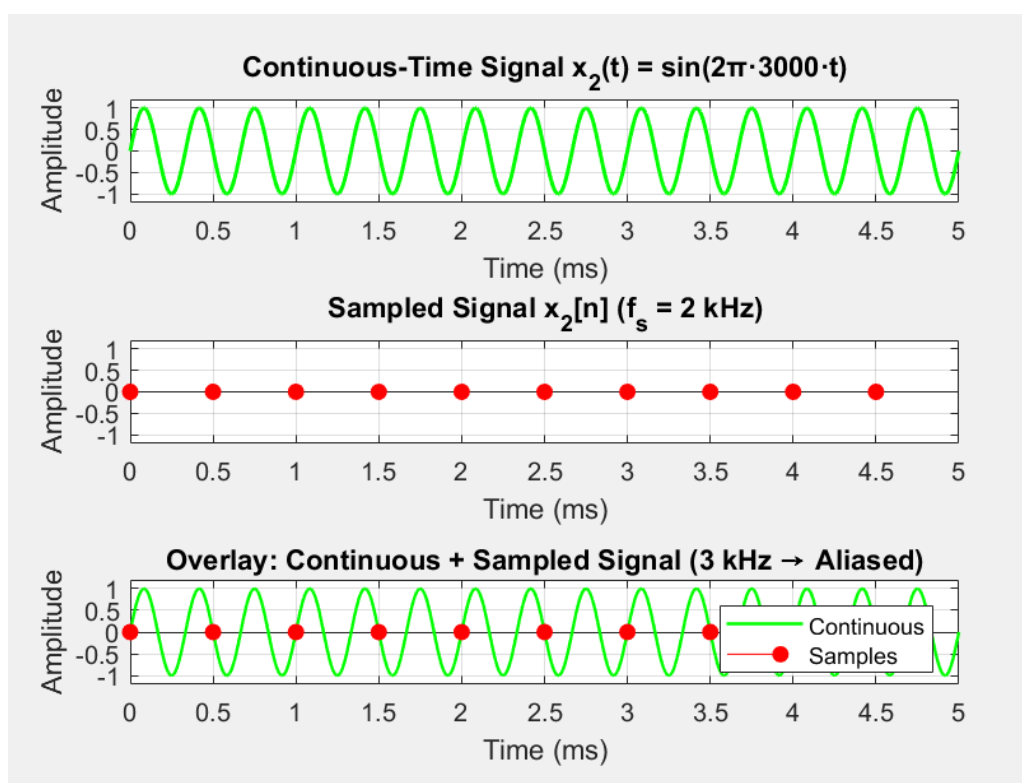
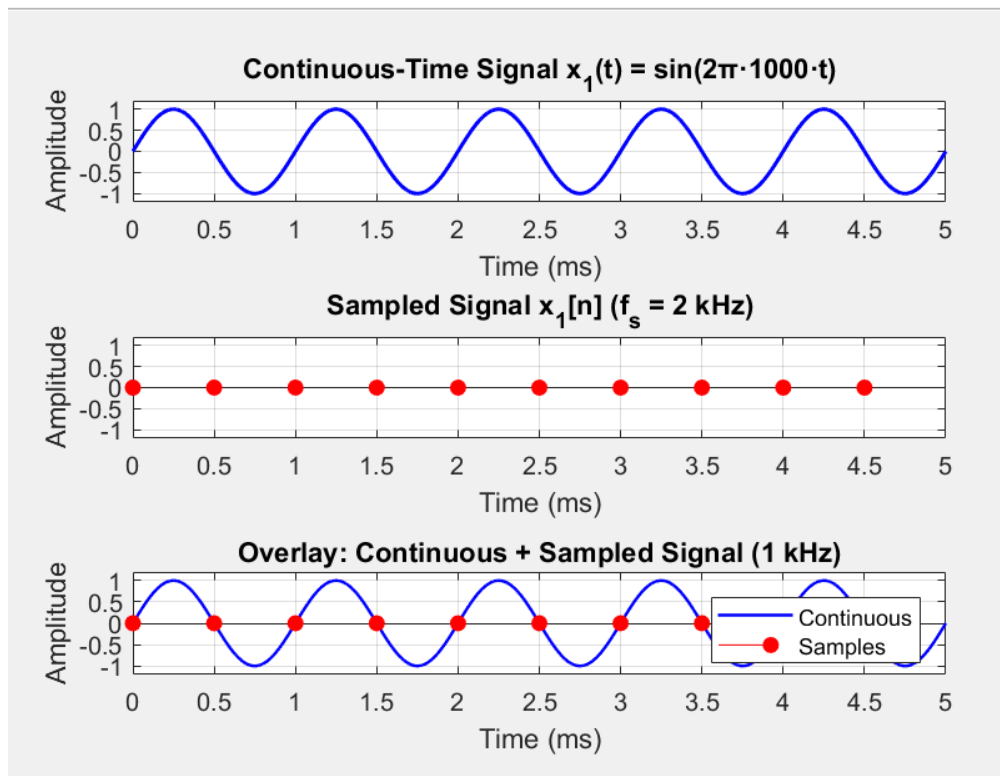
stem(n*Ts*1000, x2_n, 'ro', 'filled');
title('Sampled Signal x_2[n] (f_s = 2 kHz)');
xlabel('Time (ms)'); ylabel('Amplitude');
grid on; ylim([-1.2 1.2]); yticks(y_ticks);
xlim([0 5]); xticks(x_ticks);

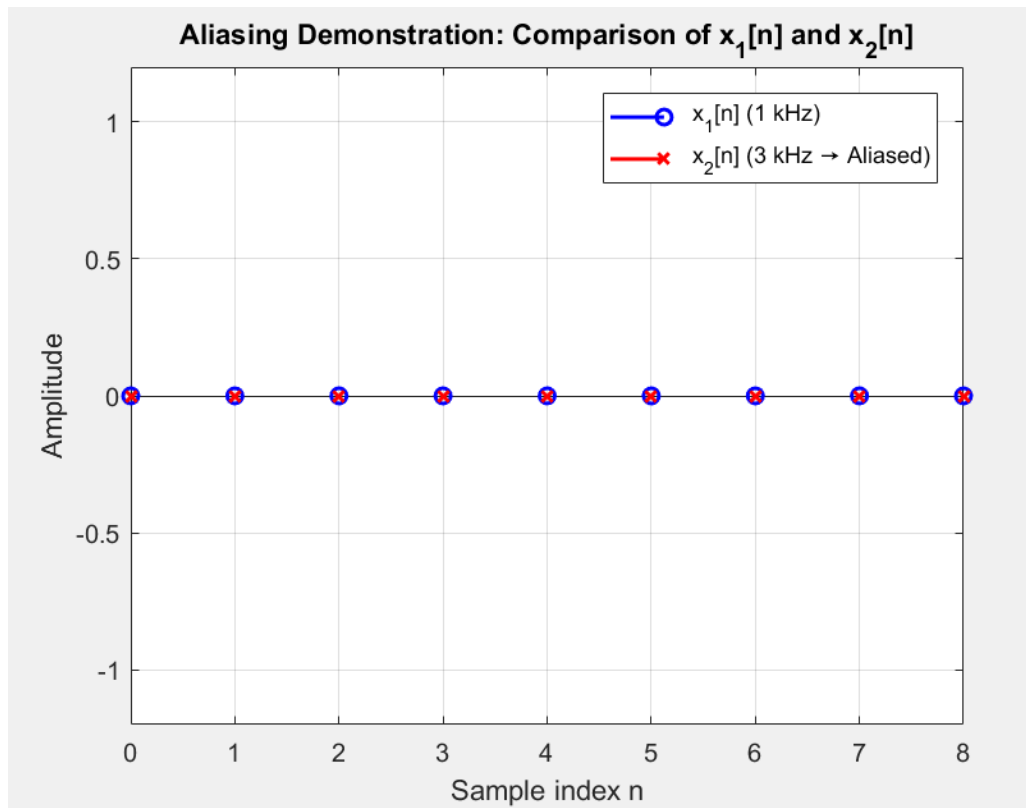
subplot(3,1,3);
plot(x_axis_ms, x2_t, 'g', 'LineWidth', 1.2); hold on;
stem(n*Ts*1000, x2_n, 'ro', 'filled');
title('Overlay: Continuous + Sampled Signal (3 kHz ? Aliased)');
xlabel('Time (ms)'); ylabel('Amplitude');
legend('Continuous', 'Samples');
grid on; ylim([-1.2 1.2]); yticks(y_ticks);
xlim([0 5]); xticks(x_ticks);

% Comparison of Sampled Sequences
figure('Name', 'Aliasing Comparison', 'NumberTitle', 'off');
stem(n, x1_n, 'bo', 'LineWidth', 1.5); hold on;
stem(n, x2_n, 'rx', 'LineWidth', 1.5);
title('Aliasing Demonstration: Comparison of x_1[n] and x_2[n]');
xlabel('Sample index n'); ylabel('Amplitude');
legend('x_1[n] (1 kHz)', 'x_2[n] (3 kHz ? Aliased)');
grid on; ylim([-1.2 1.2]); yticks(y_ticks);
xlim([0 8]);

```

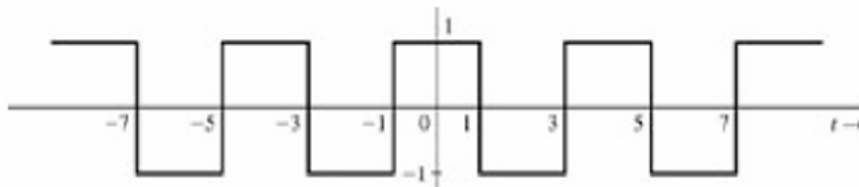
OUTPUT:





Task 02:

Fourier Series for Continuous time signals Consider the following periodic signal:



- Plot the original square wave in matlab.
- Approximate it through Fourier Series using the first $N=5,15,50$ terms (or harmonics) by plotting for each case.
- Compare the approximation with the actual square wave and observe how the approximation improves when we take summation of more and more harmonics.

Code:

```
clear;
```

```

close all;

T = 2; % Period
A = 1; % Amplitude
t_max = 3*T; % Time range to plot
N_values = [5,15, 50]; % Number of harmonics to use

% 1. Plot Original Square Wave
figure('Position', [100, 100, 1200, 800]);

% Generate square wave
t = linspace(-t_max, t_max, 1000);
square_wave = A * square(2*pi*t/T); % Standard MATLAB square wave

subplot(2,2,1);
plot(t, square_wave, 'b-', 'LineWidth', 2);
title('Original Square Wave', 'FontSize', 12);
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
ylim([-1.5*A, 1.5*A]);

% 2. Fourier Series Approximation and Frequency Domain Plots
for i = 1:length(N_values)
    N = N_values(i);

    % Fourier series coefficients for square wave
    % For odd harmonics only: a_n = (4A)/(n*pi) for n odd, 0 for n even
    t_fine = linspace(-t_max, t_max, 2000);
    approx_signal = zeros(size(t_fine));

    % Frequency domain data
    harmonics = 1:2:(2*N-1); % Odd harmonics only
    amplitudes = (4*A) ./ (pi * harmonics);
    frequencies = harmonics / T;

    % Time domain reconstruction
    for n = 1:2:(2*N-1)
        approx_signal = approx_signal + (4*A)/(pi*n) * sin(2*pi*n*t_fine/T);
    end

    % Time domain plot
    subplot(2,2,i+1);
    plot(t, square_wave, 'b--', 'LineWidth', 1, 'DisplayName', 'Original');
    hold on;
    plot(t_fine, approx_signal, 'r-', 'LineWidth', 2, 'DisplayName', ['N=' num2str(N)]);
    title(['Fourier Approximation (N=' num2str(N) ' harmonics)'], 'FontSize', 12);
    xlabel('Time (s)');

```

```

ylabel('Amplitude');
legend;
grid on;
ylim([-1.5*A, 1.5*A]);
end

% Frequency Domain Plots (Separate Figure)
figure('Position', [100, 100, 1200, 600]);

for i = 1:length(N_values)
    N = N_values(i);

    harmonics = 1:2:(2*N-1);
    amplitudes = (4*A) ./ (pi * harmonics);
    frequencies = harmonics / T;

    subplot(1,3,i);
    stem(frequencies, amplitudes, 'filled', 'LineWidth', 2);
    title(['Frequency Domain (N=' num2str(N) ' harmonics)'], 'FontSize', 12);
    xlabel('Frequency (Hz)');
    ylabel('Amplitude');
    grid on;

    % Add text annotations
    for j = 1:length(harmonics)
        text(frequencies(j), amplitudes(j), ...
            sprintf('%.2f, %.3f', frequencies(j), amplitudes(j)), ...
            'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center', ...
            'FontSize', 8);
    end
end

% 3. Combined Comparison Plot
figure('Position', [100, 100, 1000, 600]);
plot(t, square_wave, 'k', 'LineWidth', 2, 'DisplayName', 'Original Square Wave');
hold on;

for i = 1:length(N_values)
    N = N_values(i);
    t_fine = linspace(-t_max, t_max, 2000);
    approx_signal = zeros(size(t_fine));

    for n = 1:2:(2 * N - 1)
        approx_signal = approx_signal + (4 * A) / (pi * n) * sin(2 * pi * n * t_fine / T);
    end

    plot(t_fine, approx_signal, 'LineWidth', 1.5, 'DisplayName', ['N = ' num2str(N)]);
end

```

```

title('Comparison of Fourier Series Approximations', 'FontSize', 12);
xlabel('Time (s)');
ylabel('Amplitude');
legend('show');
grid on;
xlim([-t_max/2, t_max/2]); % Focus on one period
ylim([-1.5 * A, 1.5 * A]);

```

OUTPUT:

